

 UNIVERSIDAD DON BOSCO	UNIVERSIDAD DON BOSCO FACULTAD DE INGENIERÍA ESCUELA DE COMPUTACIÓN		CICLO 02-2026
	MATERIA	DESARROLLO DE SOFTWARE PARA MOVILES	GUIA DE LABORATORIO #10
	PRÁCTICA	CONSUMO DE API REST CON RETROFIT	

I. OBJETIVOS

Que al finalizar la practica el estudiante aprenda a:

- Conocer la librería Retrofit.
- Conocer el concepto de API REST.
- Consumir un API REST con Retrofit en una aplicación Android.

II. INTRODUCCIÓN TEORICA

En la actualidad consumir APIs es una tarea casi obligatoria a la hora de desarrollar una aplicación Android, sobre todo si se busca que dicha app sea útil y de mucho valor al usuario, y es que una aplicación sin conectividad con el mundo exterior es muy pobre en funcionalidades y alcance.

Afortunadamente el ecosistema de desarrollo Android es bastante rico en librerías y utilidades que hacen muy simple y sencillo implementar muchas de las características comunes en una app actual, una de esas es **Retrofit**.

Retrofit es una librería utilizada en Android para consumir servicios REST, facilita la gestión de errores, el manejo del request y response, así como el procesamiento de JSON y su conversión en objetos Java. Pero, antes de entrar de lleno a la utilización de Retrofit como tal es necesario comprender el concepto de API REST.

¿Qué es un API REST?

En términos simples API REST es un servicio que provee de las funciones necesarias para obtener información de un servicio externo, como, por ejemplo, una base de datos alojada en cualquier parte del mundo desde dentro de nuestra propia aplicación. Instagram, por ejemplo, una aplicación con millones de usuarios es inviable tener la información de cada usuario dentro de la aplicación así que para solventar el problema se utilizan servicios API REST. Siguiendo con el ejemplo, al entrar en la app el usuario se autentica (ingresando sus credenciales), esto sería el primero de los servicios, ya que se envían al servidor el usuario y contraseña y este devolverá si tenemos o no permitido el acceso a la aplicación.

Disponemos de cuatro tipos distintos de peticiones como norma general:

- **GET:** Son las peticiones más sencillas, solo nos devuelven información. Si necesitamos pasarle un parámetro a la petición será a través de la url. Es decir, si por ejemplo tenemos que hacer una petición que depende de una id (ej la identificación del usuario) la url se formaría así <https://ejemplo.com/informacion/1>, siendo 1 el parámetro que le pasamos.
- **POST:** Similar a Get pero los parámetros no se pasan por url sino por el request body, generalmente se usa para crear registros.
- **PUT:** Se suele usar para actualizar información, es decir, si pensamos en un servicio como el acceso a una base de datos, este invocará el UPDATE del usuario.
- **DELETE:** Sería el último de los cuatro que nos permitiría borrar los registros de la base de datos.

Nota: La información suele venir en dos formatos distintos, XML o JSON.

Formato JSON

JSON es un formato de texto simple, es el acrónimo de **JavaScript Object Notation**. Se trata de uno de los estándares para el traspaso de información entre plataformas, tiene una forma muy legible que permite entender su contenido sin problema. Un ejemplo sencillo sería este.

```
{
  "data": {
    "employees": [
      {
        "id": "1",
        "firstName": "Tom",
        "lastName": "Cruise",
        "photo": "https://jsonformatter.org/img/tom-cruise.jpg"
      },
      {
        "id": "2",
        "firstName": "Maria",
        "lastName": "Sharapova",
        "photo": "https://jsonformatter.org/img/Maria-Sharapova.jpg"
      },
      {
        "id": "3",
        "firstName": "Robert",
        "lastName": "Downey Jr.",
        "photo": "https://jsonformatter.org/img/Robert-Downey-Jr.jpg"
      }
    ]
  }
}
```

Antes de iniciar con la práctica, es importante saber la diferencia de utilizar R o View Binding

La Clase **R**, es un componente fundamental en el desarrollo de aplicaciones Android. Cuando se crea una aplicación, se agregan recursos como imágenes, textos y archivos XML para definir la interfaz de usuario (UI), entre otras cosas. La Clase **R** actúa como un índice o mapa que proporciona identificadores únicos para cada uno de estos recursos. Cuando se quiere acceder a un recurso, como una vista en un archivo XML, se utiliza la Clase **R** para encontrar su identificador y así poder trabajar con él en el código Java o Kotlin, como se ha venido trabajando en las guías anteriores.

Por otro lado, el **View Binding** es una característica más reciente que ofrece Android. Con **View Binding**, en lugar de referenciar vistas usando su ID a través de la Clase R, se pueden generar automáticamente clases de enlace específicas para cada archivo XML de diseño. Estas clases de enlace contienen referencias directas a todas las vistas en ese diseño, lo que permite acceder a ellas de manera más eficiente y segura en el código sin necesidad de recurrir a buscar IDs en la Clase **R**.

Ejemplo utilizando la Clase R

Se tiene un archivo XML llamado **activity_main.xml**, que contiene un **TextView** con el ID **textViewHello** y se quiere acceder a este **TextView** desde el código Kotlin utilizando la **Clase R**.

Archivo XML **activity_main.xml**:

```
<TextView
    android:id="@+id/textViewHello"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="¡Hola, mundo!" />
```

Código para acceder al **TextView** utilizando la Clase R:

```
// Importa la Clase R
import com.tu_paquete_de_aplicacion.R;

// Referencia al TextView utilizando su ID definido en XML
val textView = findViewById<TextView>(R.id.textViewHello)
```

En este caso, **R.id.textViewHello** es un identificador generado automáticamente por la Clase **R** que apunta al **TextView** en el archivo XML.

Ejemplo utilizando View Binding

Con View Binding, no se necesita hacer referencia a la Clase R para acceder a las vistas en el archivo XML. En su lugar, se generan automáticamente clases de enlace para cada archivo XML de diseño.

Archivo XML **activity_main.xml**:

```
<TextView
    android:id="@+id/textViewHello"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="¡Hola, mundo!" />
```

Código utilizando View Binding para acceder al **TextView**:

```
// Habilita View Binding en el módulo de app/build.gradle
android {
    ...
    viewBinding {
        enabled = true
    }
}
```

```
import com.tu_paquete_de_aplicacion.databinding.ActivityMainBinding

class MainActivity : AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

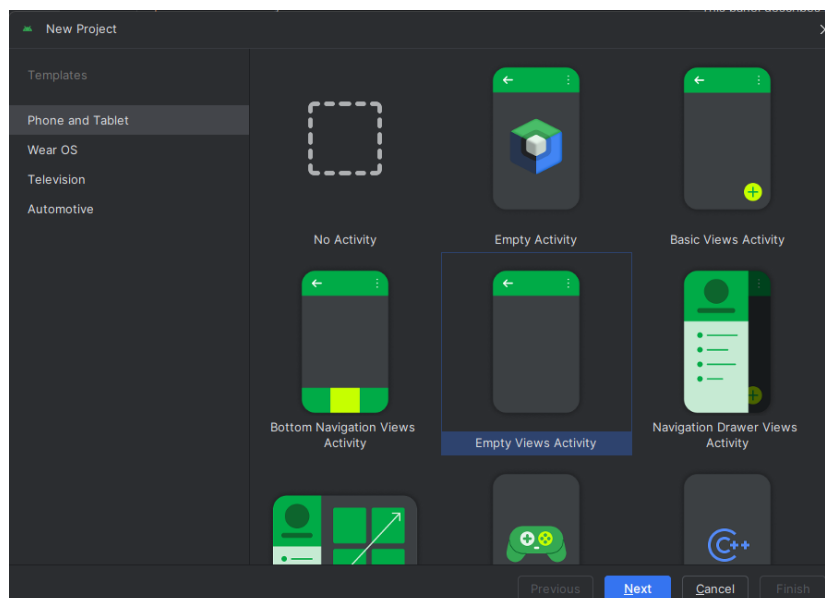
        // Ahora se puede acceder directamente a las vistas a través
        // de la clase de enlace
        val textView = binding.textViewHello
    }
}
```

ActivityMainBinding es una clase generada automáticamente que contiene referencias a todas las vistas en el archivo **XML activity_main.xml**. Entonces, puedes acceder directamente al **TextView** usando **binding.textViewHello**.

La principal diferencia radica en cómo se accede a las vistas definidas en archivos XML: con la Clase R, se utilizan identificadores generados para cada recurso, mientras que con View Binding, se accede directamente a las vistas a través de clases de enlace generadas automáticamente, lo que simplifica y hace más seguro el código.

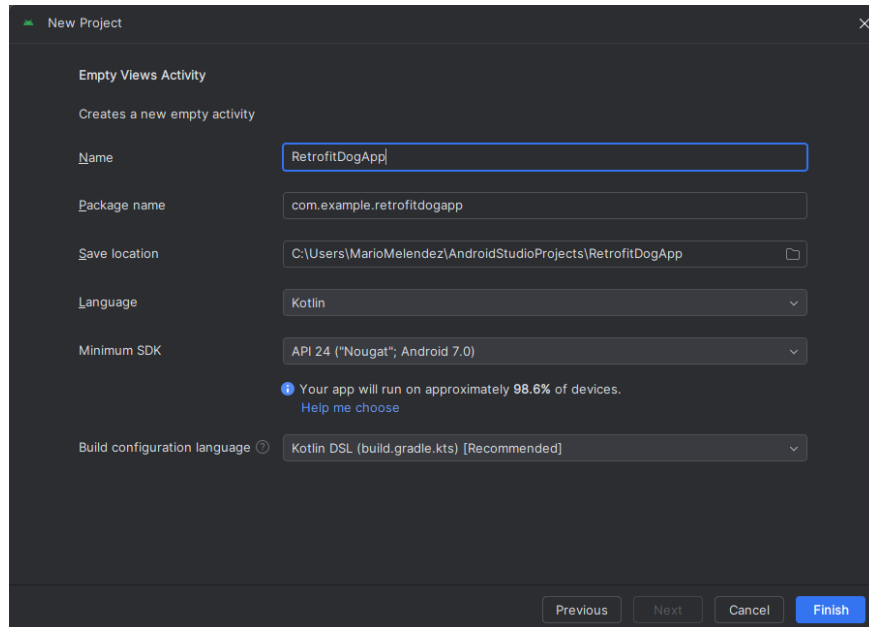
III. DESARROLLO DE LA PRÁCTICA

1. Cree un nuevo proyecto en Android Studio, asegúrese de seleccionar la plantilla **Empty Views Activity**.



2. Configure el proyecto con las siguientes características:

- **Nombre:** RetrofitDogApp.
- **Ubicación:** por defecto (o use una ubicación de su preferencia).
- **Lenguaje:** Kotlin.
- **Minimun SDK:** API 24 Nougat.



3. Edite el archivo **AndroidManifest.xml** para agregar el permiso de conectividad a internet.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET"/>

    <application>
        . . . . .
    </application>

</manifest>
```

4. Modifique el archivo **libs.versions.toml** para agregar las librerías necesarias para construir la app. Para ello, cambie la vista de su proyecto de **Android** a **Project**, posteriormente, localiza el archivo **libs.versions.toml** en la carpeta **gradle**.

- **Picasso:** Esta librería permite transformar las urls en imágenes.
- **Retrofit 2:** Librería encargada del consumo de las API.
- **Retrofit 2 Converter Gson:** complemento que simplificará el proceso de pasar un JSON a una clase Kotlin.

En la sección **[versions]**, defina las siguientes versiones de las herramientas.

- **picasso** = "2.8"
- **retrofit** = "2.9.0"
- **gson** = "2.9.0"

En la sección **[libraries]** defina las librerías con sus respectivos nombres y referencias a las versiones previamente escritas.

- **picasso** = { module = "com.squareup.picasso:picasso", version.ref = "picasso" }
- **retrofit** = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }
- **retrofit-converter-gson** = { module = "com.squareup.retrofit2:converter-gson", version.ref = "gson" }

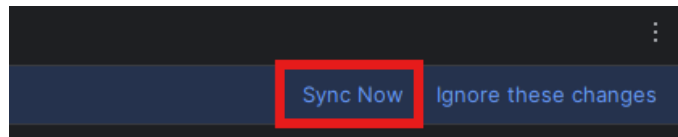
```
1  [versions]
2  android = "8.11.2"
3  kotlin = "2.0.21"
4  coreKtx = "1.17.0"
5  junit = "4.13.2"
6  junitVersion = "1.3.0"
7  espressoCore = "3.7.0"
8  appcompat = "1.7.1"
9  material = "1.12.0"
10 activity = "1.10.1"
11 constraintlayout = "2.2.1"
12 picasso = "2.8"
13 retrofit = "2.9.0"
14 gson = "2.9.0"
15
16 [libraries]
17 androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
18 junit = { group = "junit", name = "junit", version.ref = "junit" }
19 androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
20 androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
21 androidx-appcompat = { group = "androidx.appcompat", name = "appcompat", version.ref = "appcompat" }
22 material = { group = "com.google.android.material", name = "material", version.ref = "material" }
23 androidx-activity = { group = "androidx.activity", name = "activity", version.ref = "activity" }
24 androidx-constraintlayout = { group = "androidx.constraintlayout", name = "constraintlayout", version.ref = "constraintlayout" }
25 picasso = { module = "com.squareup.picasso:picasso", version.ref = "picasso" }
26 retrofit = { module = "com.squareup.retrofit2:retrofit", version.ref = "retrofit" }
27 retrofit-converter-gson = { module = "com.squareup.retrofit2:converter-gson", version.ref = "gson" }
```

5. Modifique el archivo **build.gradle.kts (Module :app)** para agregar los nombres definidos previamente en el archivo **libs.versions.toml** para implementar las dependencias.

```
dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.appcompat)
    implementation(libs.material)
    implementation(libs.androidx.activity)
    implementation(libs.androidx.constraintlayout)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)

    // Picasso, retrofit y gson
    implementation(libs.picasso)
    implementation(libs.retrofit)
    implementation(libs.retrofit.converter.gson)
}
```

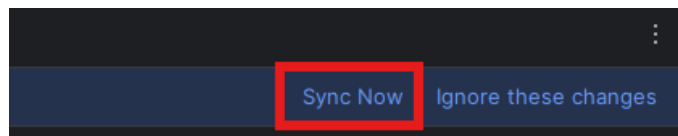
6. Sincronice el proyecto con Gradle dando clic en la opción **Sync Now** o en caso de que no aparezca, cierre y abra nuevamente el proyecto.



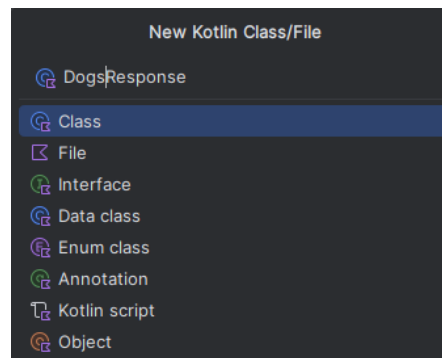
7. Edite nuevamente el archivo **build.gradle.kts (Module :app)** para utilizar el View Binding en la aplicación, para ello, en la sección **android** agregue las siguientes líneas de código.

```
6  android {
7      namespace = "com.example.retrofitdogapp"
8      compileSdk = 36
9
10     buildFeatures {
11         dataBinding = true
12         viewBinding = true
13     }
14 }
```

8. Sincronice nuevamente el proyecto.



9. Sobre el paquete de la aplicación agregue una nueva clase de **Kotlin** con el nombre **DogsResponse**, modifique el código de la siguiente manera.



```
import com.google.gson.annotations.SerializedName

// Definición de la clase DogsResponse que representa la respuesta de la API
class DogsResponse {

    // Campo que representa el estado de la respuesta de la API
    @SerializedName("status")
    private var status: String? = null
}
```

```

// Campo que representa la lista de URLs de imágenes
@SerializedName("message")
private var images: List<String?>? = null

// Obtiene el estado de la respuesta
fun getStatus(): String? {
    return status
}

// Establece el estado de la respuesta
fun setStatus(status: String?) {
    this.status = status
}

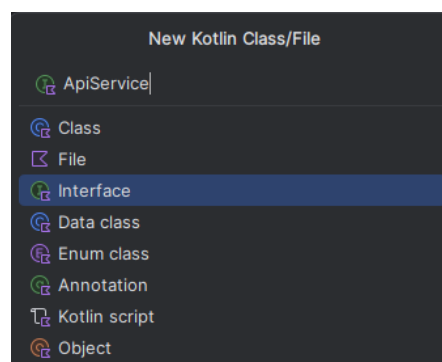
// Obtiene la lista de URLs de imágenes
fun getImages(): List<String?>? {
    return images
}

// Establece la lista de URLs de imágenes
fun setImages(images: List<String?>?) {
    this.images = images
}
}

```

NOTA: esta clase representa la respuesta de la API.

10. Agregue una interfaz donde se especifique el nombre del método (no se define funcionalidad) que se encargara de hacer la llamada al API utilizando **Retrofit**. Modifique el código de la siguiente manera.



```

package com.example.retrofitdogapp

import retrofit2.Call
import retrofit2.http.GET
import retrofit2.http.Path

interface ApiService {
    /*
    Obtiene imágenes de perros por raza

```

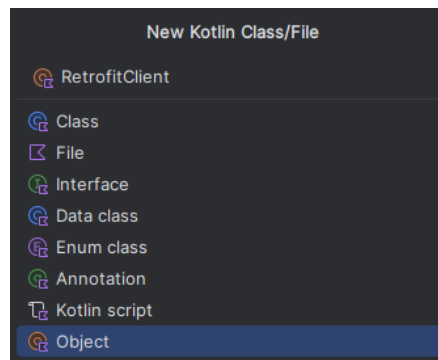
```

    @GET indica que es una solicitud HTTP GET

    @Path se utiliza para agregar el valor de la variable raza a la URL
    */
    @GET("{raza}/images")
    fun getDogsByBreed(@Path("raza") raza: String?): Call<DogsResponse?>?
}

```

11. Agregue un nuevo objeto con el nombre **RetrofitClient**, modifique el código según se muestra a continuación.



```

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitClient {

    private const val BASE_URL = "https://dog.ceo/api/breed/"

    val instance: ApiService by lazy {
        val retrofit = Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()

        retrofit.create(ApiService::class.java)
    }
}

```

12. Sobre el directorio **res/layout** agregue un nuevo archivo de recursos con el nombre **item_dog.xml** y modifique el contenido según se muestra a continuación.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.cardview.widget.CardView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="320dp"
    android:layout_margin="16dp"
    android:background="@color/black"

```

```

        app:cardCornerRadius="16dp">

        <ImageView
            android:id="@+id/ivDog"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:scaleType="centerCrop"/>

</androidx.cardview.widget.CardView >

```

13. Esta app utiliza **RecyclerView** para mostrar la lista de imágenes devueltas por el API, por lo que es necesario agregar clases que ayuden a la gestión de los ítems de la lista. El primer elemento es una clase de tipo **ViewHolder**, para ello agregue una nueva clase con el nombre **DogViewHolder** y modifique el código de la siguiente manera.

```

import android.view.View
import androidx.recyclerview.widget.RecyclerView
import com.example.retrofitdogapp.databinding.ItemDogBinding
import com.squareup.picasso.Picasso

class DogViewHolder(view: View) : RecyclerView.ViewHolder(view) {

    // Declaración de una instancia de ItemDogBinding para acceder a las vistas en
    // el diseño del elemento
    // Vinculación de la vista a la clase de enlace generada para el diseño del
    // elemento
    private val itemDogBinding: ItemDogBinding = ItemDogBinding.bind(view)

    // Vincula la URL de la imagen al ImageView en el diseño del elemento
    fun bind(imageUrl: String?) {
        // Carga la imagen desde la URL usando Picasso y la muestra en el ImageView
        ivDog en el diseño del elemento
        Picasso.get().load(imageUrl).into(itemDogBinding.ivDog)
    }
}

```

14. Agregue una nueva clase con el nombre **DogAdapter** y modifique el código como se muestra continuación.

```

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.recyclerview.widget.RecyclerView

class DogAdapter(private val images: List<String>?) : RecyclerView.Adapter<Dog-
ViewHolder>() {

    // Se llama cuando RecyclerView necesita un nuevo ViewHolder para representar
    // un elemento
    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): Dog-
ViewHolder {
        val view: View = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_dog, parent, false)
    }
}

```

```

        return DogViewHolder(view)
    }

    // Se llama para mostrar los datos en una posición específica
    override fun onBindViewHolder(holder: DogViewHolder, position: Int) {
        holder.bind(images!![position])
    }

    // Devuelve el número total de elementos en el conjunto de datos
    override fun getItemCount(): Int {
        return images?.size ?: 0
    }
}

```

15. Modifique el archivo **activity_main.xml** de la siguiente forma.

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <androidx.appcompat.widget.SearchView
        android:id="@+id/searchDogs"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        app:layout_constraintVertical_bias="0"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/listDogs"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toBottomOf="@+id/searchDogs"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintBottom_toBottomOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

16. Modifique el código de la clase **MainActivity** de la siguiente manera.

```
import android.os.Bundle
import android.widget.Toast
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.widget.SearchView
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat
import androidx.recyclerview.widget.LinearLayoutManager
import com.example.retrofitdogapp.databinding.ActivityMainBinding
import retrofit2.Call
import retrofit2.Callback
import retrofit2.Response
import java.util.Locale

class MainActivity : AppCompatActivity(), SearchView.OnQueryTextListener {

    private lateinit var binding: ActivityMainBinding
    private lateinit var dogAdapter: DogAdapter
    private var images: MutableList<String> = ArrayList()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        // Inflar el diseño de la actividad y establecerlo como el contenido de la
vista
        binding = ActivityMainBinding.inflate(layoutInflater)
        setContentView(binding.root)

        enableEdgeToEdge()

        ViewCompat.setOnApplyWindowInsetsListener(binding.root) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, system-
Bars.bottom)
            insets
        }

        initRecyclerView()
        binding.searchDogs.setOnQueryTextListener(this as SearchView.OnQueryText-
Listener)
    }

    private fun initRecyclerView(){
        dogAdapter = DogAdapter(images)
        binding.listDogs.layoutManager = LinearLayoutManager(this)
        binding.listDogs.adapter = dogAdapter
    }
}
```

```

    private fun searchByName(raza: String) {
        val batch: Call<DogsResponse?> = RetrofitClient.instance.getDogs-
ByBreed(raza)

        batch?.enqueue(object : Callback<DogsResponse?> {

            override fun onResponse(call: Call<DogsResponse?>, response: Re-
sponse<DogsResponse?>) {
                if (response.isSuccessful && response.body() != null) {
                    val responseImages: List<String> = re-
sponse.body()!!.getImages() as List<String>
                    images.clear()
                    images.addAll(responseImages)
                    dogAdapter.notifyDataSetChanged()
                }
            }

            override fun onFailure(call: Call<DogsResponse?>, t: Throwable) {
                showError()
            }
        })
    }

    private fun showError(){
        Toast
            .makeText(this, "Ocurrió un error", Toast.LENGTH_SHORT)
            .show()
    }

    override fun onQueryTextChange(query: String): Boolean {
        if(query.isNotEmpty()){
            searchByName(query.lowercase(Locale.getDefault()))
        }
        return true
    }

    override fun onQueryTextSubmit(newText: String?): Boolean {
        return true
    }
}

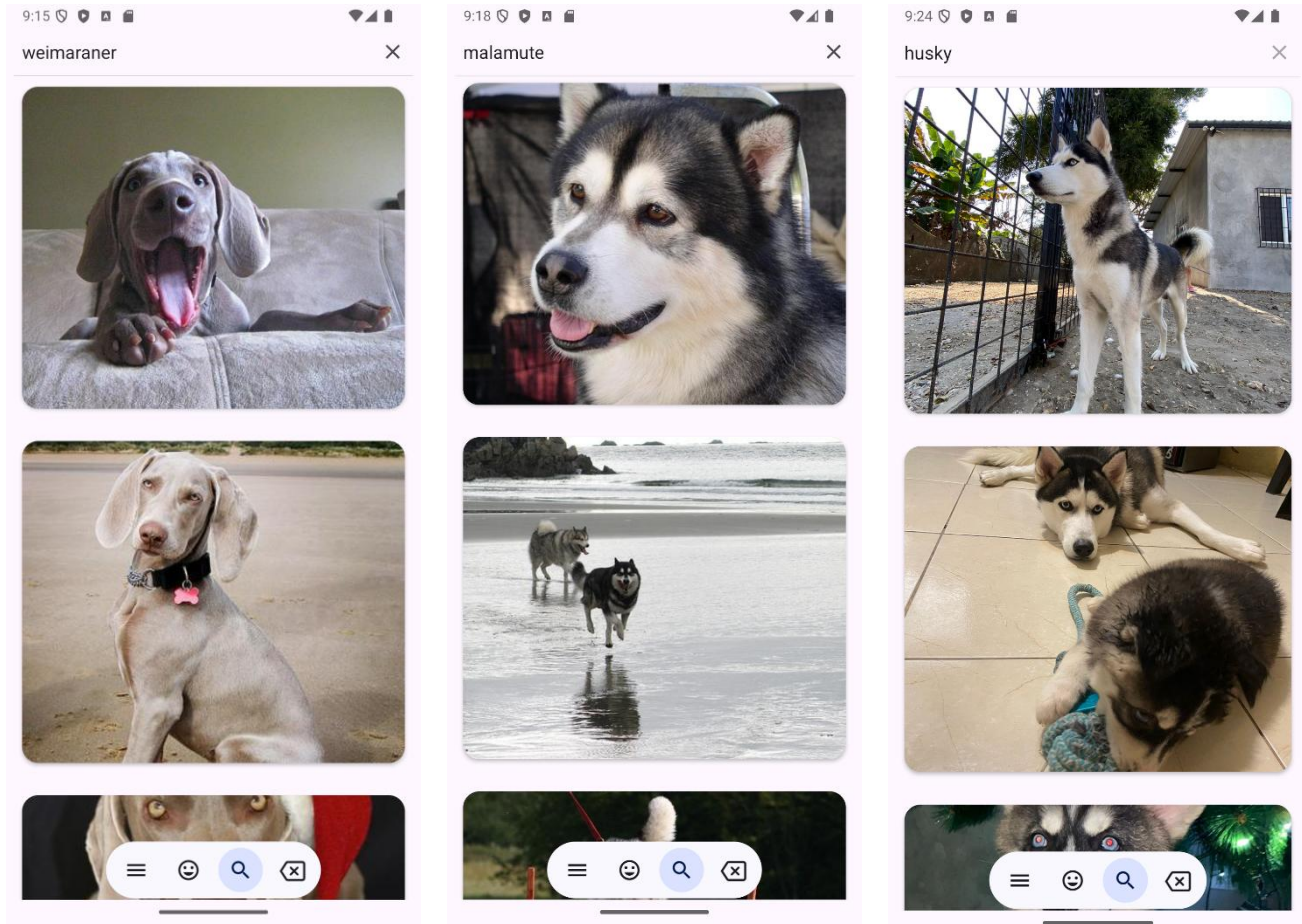
```

NOTA:

1. La clase **MainActivity** implementa **SearchView.OnQueryTextListener** para escuchar eventos del campo de búsqueda (texto cambiado o enviado).
2. Métodos de búsqueda **onQueryTextChange** se ejecuta cada vez que el texto cambia y si el texto no está vacío, llama a **searchByName()** con la raza en minúsculas.

3. El método **searchByName()** es el encargado de llamar al método de Retrofit que busca perros por raza.
- Ejecuta la llamada de forma asíncrona.
 - En **onResponse**: si hay éxito, actualiza la lista de imágenes y notifica al adaptador.
 - En **onFailure**: muestra un error con Toast.

17. Ejecute la app y compruebe su funcionamiento, escriba el nombre de una raza en la caja de texto y observe la carga de imágenes según el parámetro especificado.



IV. DISCUSIÓN DE RESULTADOS

Crea una aplicación Android que se conecte al API REST de **Github** que permita buscar por nombre de usuario (**username**) y muestre un listado de sus repositorios públicos.

Endpoint: <https://api.github.com/users/{username}/repos>

Ejemplo de respuesta:

```
[
  {
    "id": 1,
    "node_id": "MDEwOlJlcG9zaXRvcnkx",
    "name": "grit",
    "full_name": "mojombo/grit",
    "owner": {
      "login": "mojombo",
      ...
    },
    "private": false,
    ...
  },
  ...
]
```

V. BIBLOGRAFÍA

- Documentación Oficial Android
<https://developer.android.com/develop/ui/views/layout/recyclerview?hl=es-419#kotlin>
- Documentación Oficial Retrofit
<https://square.github.io/retrofit/>